

# Exercise 3 — Build it with a durable working method

*A self-paced companion using real GitHub Copilot custom agents*

*Stand-alone version of Exercise 3, for working on your own with **real GitHub Copilot custom agents** — not a simulation. The workspace bundled alongside this document already has the skill and the three agents configured as files; there is nothing to install or paste, only to open and confirm.*

## What you are building

The exact same domain as Exercise 2, from the exact same two specifications — nothing about the software changes. What changes is the *process*: instead of one long conversation, the way you use the assistant becomes something you can inspect, approve, and reuse, not just something that happened once in a chat window.

## Before you start

1. Unzip the accompanying `exercise-3-copilot-workspace` folder and open **the whole folder** in VS Code — not a single file.
2. Confirm Java 21, Maven, Git, and your GitHub Copilot licence are available.
3. Open Copilot Chat and run **Chat: Open Customizations**. Confirm you can see:
  - Specification Planner, Domain Implementer, Conformance Reviewer;
  - `/create-plan`, `/implement-approved-plan`, `/verify-conformance`; and
  - the `implement-sell-stocks` skill.

If any item is missing, your Copilot licence or organisation policy does not currently support custom agents or skills — check that before continuing; nothing later in this document works without it.

The two specifications are already in `docs/spec/` inside the workspace — you do not need to place them yourself:

- `sell-stocks-spec.md` — behaviour: preconditions, the FIFO rule, the money definitions, and acceptance criteria AC-01 ... AC-24.
- `domain-class-diagram.puml` — structure: classes, fields, methods, visibility, and relationships.

## Why this exercise exists

Exercise 2 made the **behaviour** durable: it lives in a spec file instead of your memory of a conversation. But the **process** you used to turn that spec into code — the questions you asked first, the order you built things in, who checked the result and how — was still invisible. It happened once, inside one conversation, and vanished the moment the chat ended.

This exercise does for the *working method* what Exercise 2 did for the *domain*: it takes something that used to live only in a conversation and turns it into something you can read, review, and reuse — a **skill** for the repeatable method, and **role-scoped agents** for who is allowed to do what. Here, both are real, checked-in files: `.agents/skills/implement-sell-stocks/` and `.github/agents/*.agent.md`, already in the workspace you opened.

### A concrete failure this catches

Picture Exercise 2 again. The specification never says what should happen at some edge case the acceptance criteria don't cover — say, a sale priced exactly at a lot's own purchase price. In one continuous conversation, the assistant can quietly pick an answer, write the code for it, then write a test that agrees with the code it just wrote — and you would very likely never notice, for the same reason Exercise 1's self-graded tests never failed: the actor that made the decision and the actor that checked it were the same conversation, with the same blind spot.

Exercise 3 puts two structural obstacles in that path, and here they are enforced by Copilot itself, not just by discipline:

1. **The Specification Planner agent has no edit tool.** It is configured to read and propose only — it cannot touch a file even if it tried. If it silently invents behaviour, that invention shows up as a line in `plan.md` you are explicitly asked to approve, not as a fait accompli buried in a diff you skim past.
2. **The Conformance Reviewer agent has no edit tool either**, and is selected as an independent pass over the finished implementation. It checks the code against the specification and the diagram directly, not against “does this look like what I would have written” — because Copilot never let it write anything in the first place.

Neither obstacle *guarantees* the assumption gets caught. What they guarantee is that it has to surface somewhere reviewable, instead of staying invisible inside one uninterrupted train of thought.

### Exercise 2 vs. Exercise 3

| DIMENSION     | EXERCISE 2                        | EXERCISE 3                                 |
|---------------|-----------------------------------|--------------------------------------------|
| Domain inputs | Two specifications                | The same two specifications                |
| Process       | One conversation, start to finish | Three role-scoped agents, invoked in order |

| DIMENSION             | EXERCISE 2                                | EXERCISE 3                                                                 |
|-----------------------|-------------------------------------------|----------------------------------------------------------------------------|
| Planning evidence     | Whatever you remember from the chat       | <code>plan.md</code> — written, dated, approved                            |
| Who checks the result | Often the same conversation that built it | The Conformance Reviewer agent, which has no edit tool at all              |
| Reuse next time       | Retype the prompt                         | The skill and agent files stay in the repository, discovered automatically |

### Is it worth the overhead? Be honest about this

At this kata's size, Exercise 3 is genuinely more work than Exercise 2's thirty-line prompt — three agent invocations and a written approval instead of one. That overhead does not pay for itself on a task this small, and “Worth trying afterwards” at the end asks you to feel that directly.

It starts paying off at a different scale than this kata: when a change is large or risky enough that you need to *prove*, after the fact, who approved what before it was built — closer to how a bank already treats a pull-request approval or a change record than it might first appear. It also pays off across time: a skill and a set of agents, written once, get reused on the next feature; a prompt typed once gets forgotten.

Hold both of those in mind as you work through the rest of this document — the four ideas below are what carries that value, not the paperwork itself.

### The four ideas — and why each one matters

1. **A skill is a versioned, repeatable method** — not a one-off prompt you retype each time. It lives in `.agents/skills/implement-sell-stocks/SKILL.md`, reviewable like any other file in the repository, and Copilot loads it automatically when it's relevant.
2. **Role separation is a permission boundary, not a suggestion.** The Specification Planner has no edit tool at all, so it cannot quietly start implementing before you've approved anything; the Conformance Reviewer has no edit tool either, so it cannot “fix while reviewing” and blur checking into doing.
3. **A human checkpoint gates planning → implementation.** Nothing gets built until you've read the plan, changed at least one thing yourself, and written down that you approve it — turning approval into a real decision instead of a reflexive “looks good.”
4. **Independent review happens before any fix.** The reviewer reports findings first; you decide what goes back to the implementer — keeping “did we build the right thing” separate from “do I like what got built.”

## Technical constraints

- Java 21, Maven, a standalone project with its own `pom.xml`, no parent.
- JUnit 5 only, in test scope. No Spring, no persistence, no REST layer.
- Package root: `com.neueda.portfolio.domain`.
- Every monetary amount as `BigDecimal`, scale 2, `HALF_UP` — never `double` or `float`.

These are already encoded in `AGENTS.md` and the agent files — you don't need to repeat them yourself; they're here so you know what to check for while reviewing.

## Step 1 — Plan

Run this in Copilot Chat:

COPY THIS BLOCK

```
/create-plan
```

The Specification Planner reads both specifications but cannot edit your files — it has no edit tool. When it returns a proposed plan:

1. copy the proposal into `plan.md`;
2. change or refine **at least one item yourself** — a split task, an added risk, a corrected dependency order, a criterion the proposal under-specified;
3. confirm AC-01 through AC-24 all appear in the coverage table;
4. change the plan status to `approved`; and
5. fill in the Human checkpoint block: `Decision: approved`, your name, the date.

Do not continue until this file says `approved` in your own words, not the assistant's. This step is the point of the exercise — it is what makes approval a review act instead of a rubber stamp.

## Step 2 — Implement

Run:

COPY THIS BLOCK

```
/implement-approved-plan
```

This generates `pom.xml`, `src/main`, and `src/test` following the approved plan. Review each checkpoint and proposed diff — do not approve everything automatically. Run `mvn test` yourself once it reports done; don't take “the tests pass” on trust from the transcript.

## Step 3 — Independent review

Run:

COPY THIS BLOCK

```
/verify-conformance
```

The Conformance Reviewer has no edit tool, so it can only report findings, not fix them. Read the findings and classify each one: a verified defect, a suspected risk needing more evidence, an optional improvement outside the specification, or a false positive. Only findings **you** accept go back to the implementer.

## Step 4 — Final check

Run this yourself from the workspace root:

COPY THIS BLOCK

```
mvn test
```

Expected result: exactly **36 tests**, no failures or errors. If Bash is available, you can also run `.agents/skills/implement-sell-stocks/scripts/verify-workspace.sh` for an additional mechanical check.

## Completion contract

| # | REQUIREMENT                                                                             |
|---|-----------------------------------------------------------------------------------------|
| 1 | <code>mvn test</code> passes with exactly 36 tests, no failures or errors               |
| 2 | AC-01 through AC-24 are traceable in test names or display names                        |
| 3 | The implementation matches the class diagram — nothing extra, nothing missing           |
| 4 | <code>plan.md</code> was approved, in your own words, before any implementation started |
| 5 | The Conformance Reviewer ran and reported before any fix                                |
| 6 | Only findings you accepted were sent back for correction                                |
| 7 | <code>plan.md</code> still accurately describes what was actually built                 |

A green test run alone does not satisfy this contract — items 4 through 6 are about the *process*, and nothing in the code proves they happened.

## Worth trying afterwards

- **Ask the Domain Implementer to also review its own work**, instead of switching to the Conformance Reviewer agent, and watch it agree with itself. It's the same blind spot as Exercise 1's self-written tests, wearing a different costume — an evaluator that already committed to an answer rarely reverses itself.
- **Approve a plan with a task deliberately left out**, then run `/implement-approved-plan`. Does it notice the gap and ask, or build past it silently?
- **Count the overhead**. Exercise 2 was one prompt of about thirty lines. This exercise was three agent invocations plus a written, human-approved checkpoint. Was that worth it at this kata's size? At what size would it be?

## What's next

The domain became durable in Exercise 2. The working method became durable here. Exercise 4 keeps both and changes what they get pointed at: the same unchanged domain, now wrapped in a REST API and a real database.